

Configurando o Harness na Prática: settings.json, Hooks e Permissions & Construindo Tools e Servidores MCP: Schemas e Blindagem contra Tool Poisoning — Um Recorte de AI Driven Development: Do Zero ao Deploy

Heverton Eduardo Peres

RESUMO

Este artigo apresenta um recorte investigativo sobre a governança técnica de agentes de codificação, derivado do dossiê da obra *AI Driven Development: Do Zero ao Deploy*. O problema de pesquisa parte da distância entre o que um agente de IA tenta fazer e o que efetivamente lhe é permitido fazer, distância preenchida por quatro camadas de controle tratadas de forma fragmentada na literatura: configuração do harness via settings.json, hooks e permissions; construção de tools e servidores MCP (Model Context Protocol) com schemas validados e blindagem contra tool poisoning; disciplina de economia severa de tokens (caveman, RTK-memory, lean-ctx, headroom) como condição de sustentabilidade de sessões agênticas estendidas; e integração de agentes em pipelines de CI/CD sob portão de aprovação humana antes do deploy em produção. Metodologicamente, o artigo não realiza coleta de dados original, mas sintetiza, por recuperação indexada e triangulação de fontes, mais de vinte referências técnicas por seção extraídas do dossiê-mãe, incluindo documentação oficial de fornecedor, catálogos de ataque do OWASP e da Cloud Security Alliance, pré-publicações em arXiv e relatos de prática de mercado. Os resultados mostram que as quatro camadas funcionam como anteparas redundantes: a falha de qualquer uma reabre exatamente a lacuna de exposição que as demais foram desenhadas para fechar. Conclui-se que a governança de agentes de codificação deve ser tratada como requisito único de arquitetura, e não como itens de configuração independentes, sob pena de a antepara mais fraca determinar o nível de exposição real de toda a esteira de desenvolvimento assistido por IA.

Palavras-chave: harness agêntico, Model Context Protocol, tool poisoning, economia de tokens, CI/CD agêntico.

ABSTRACT

This article presents an investigative excerpt on the technical governance of coding agents, derived from the dossier of the book *AI Driven Development: Do Zero ao Deploy*. The research problem stems from the gap between what an AI agent attempts to do and what it is actually permitted to do, a gap filled by four control layers that the literature typically treats in a fragmented way: harness configuration via settings.json, hooks and permissions; building tools

and MCP (Model Context Protocol) servers with validated schemas and hardening against tool poisoning; a discipline of severe token economy (caveman, RTK-memory, lean-ctx, headroom) as a condition for the sustainability of extended agentic sessions; and integrating agents into CI/CD pipelines under a human approval gate before production deployment. Methodologically, the article performs no original data collection, instead synthesizing, through indexed retrieval and source triangulation, more than twenty technical references per section drawn from the parent dossier, including official vendor documentation, OWASP and Cloud Security Alliance attack catalogs, arXiv preprints, and market practice reports. The results show that the four layers function as redundant bulkheads: the failure of any one reopens exactly the exposure gap the others were designed to close. The article concludes that coding-agent governance must be treated as a single architectural requirement, not as independent configuration items, since the weakest bulkhead otherwise determines the actual exposure level of the entire AI-assisted development pipeline.

Keywords: agentic harness, Model Context Protocol, tool poisoning, token economy, agentic CI/CD.

1 INTRODUÇÃO

1.1 Contextualização do problema

A adoção de harnesses agênticos de codificação — ferramentas que colocam um modelo de linguagem no centro de um ciclo de leitura, edição e execução de comandos sobre um repositório real — deixou de ser experimento de laboratório para se tornar prática corrente de engenharia de software. Levantamentos de mercado de 2026 registram que 76,6% das organizações já usam IA ativamente em desenvolvimento, com 20,4% adicionais avaliando adoção (FUTURUM, 2026), e a Forrester descreve a migração de assistentes de código pontuais para agentes orquestrados de ciclo de vida completo (SDLC) como a tendência dominante do ano (FORRESTER, 2026). Esse deslocamento levanta um problema de pesquisa concreto: um harness como o Claude Code decide o que é permitido — cada ferramenta possui seu próprio portão de permissão, verificado contra um pipeline de regras antes de qualquer execução —, enquanto o modelo decide apenas o que tentar (ANTHROPIC, 2026). A distância entre “o que o agente tenta” e “o que o agente pode” é preenchida por configuração explícita: o arquivo `settings.json`, os hooks determinísticos, os servidores MCP (Model Context Protocol) e as próprias práticas de economia de contexto que decidem quanto e o quê o modelo enxerga a cada chamada.

Essa distância é também o ponto de maior risco documentado na literatura técnica recente. O OWASP catalogou o *MCP Tool Poisoning* como um vetor de injeção indireta de prompt em que instruções maliciosas embutidas na descrição de uma ferramenta MCP sequestram o raciocínio do agente no momento do registro da ferramenta, antes mesmo de qualquer chamada real (OWASP, 2026). A Microsoft documenta o mesmo padrão de ataque como problema estrutural do protocolo, não falha isolada de implementação (MICROSOFT, 2026). Willison (2026) demonstrou publicamente que a arquitetura do MCP, ao combinar acesso a dados privados, exposição a conteúdo não confiável e capacidade de comunicação externa, reproduz a “trifecta letal” de vulnerabilidades já conhecida em agentes de IA. No extremo oposto do pipeline, o paper “GitInject” documenta ataques reais de injeção de prompt em pipelines de CI/CD alimentados por IA, explorando títulos de *pull request*, *issues* e comentários de repositório como superfície de ataque (ARXIV, 2026).

1.2 Objetivo do recorte

Este artigo tem por objetivo sintetizar, de forma integrada, quatro frentes de governança técnica de agentes de codificação que a literatura recente trata de modo predominantemente fragmentado: (i) a configuração prática do harness via `settings.json`, hooks e permissions; (ii) a construção de tools e servidores MCP com schemas validados e blindagem contra tool poisoning; (iii) a disciplina de economia severa de tokens como pré-condição de sustentabilidade operacional de agentes de longa duração; e (iv) a integração desses agentes em pipelines de CI/CD sob um portão de aprovação humana antes do deploy em produção. O recorte não propõe uma técnica nova, mas articula essas quatro frentes como uma única pilha de controle, na qual cada camada cobre a lacuna de exposição deixada pelas demais.

1.3 Justificativa e relevância

A justificativa para tratar essas quatro frentes como um objeto de estudo único decorre de uma constatação recorrente na literatura: nenhuma camada isolada é suficiente. A abordagem de segurança do Claude Code é descrita como multicamadas — permissions como aplicação diária, *managed settings* como política corporativa, hooks como aplicação determinística e controles de MCP como governança de ferramentas —, com a analogia recorrente de tratar o agente “como um funcionário júnior novo com acesso root”: dar apenas o acesso necessário, observar constantemente e checar duas vezes quando a ação for arriscada (GENERAL, 2026). A mesma lógica de anteparas redundantes aparece no fechamento do ciclo: práticas de segurança recomendadas para deploy incluem credenciais de curta duração, privilégio mínimo, limite de gasto de tokens e a manutenção de um portão de aprovação humana entre a mudança de código do agente e o deploy em produção — o agente abre o *pull request*, o CI valida, um humano aprova o merge e o pipeline de deploy dispara automaticamente, nunca o contrário (TEAMVOY, 2026). Relatos de equipes técnicas de mercado confirmam esse padrão em produção: DeployHQ, Spacelift e Augment Code documentam pipelines reais com agentes revisando PRs e reparando testes, sempre com aprovação humana antes de produção (DEPLOYHQ, 2026; SPACELIFT,

2026; AUGMENT, 2026). Finalmente, a camada de economia de tokens é justificada por um argumento de custo estrutural: em fluxos de agente estendidos, o processamento de contexto domina o custo total, de modo que a curadoria do que entra na janela de contexto — e não apenas a escolha de palavras do prompt — determina diretamente a viabilidade econômica de operar agentes de codificação em escala (AGENTA, 2026; REDIS, 2026). Compreender essas quatro frentes como uma pilha única, e não como tópicos avulsos, é o que este artigo se propõe a demonstrar.

2 METODOLOGIA

2.1 Natureza do recorte e corpus documental

Este artigo é um recorte investigativo derivado de um dossiê técnico mais amplo, produzido para a obra-mãe *AI Driven Development: Do Zero ao Deploy*, e não um estudo experimental com coleta de dados original. A opção metodológica é a de revisão analítico-integrativa de literatura técnica, aplicada a quatro blocos temáticos do dossiê-mãe que correspondem aos Capítulos 7 a 10 daquela obra: configuração prática do harness (settings.json , hooks, permissions), construção de tools e servidores MCP com blindagem contra tool poisoning, economia severa de tokens (caveman, RTK-memory, lean-ctx, headroom) e integração de agentes em CI/CD com portão de aprovação humana. O corpus consolidado que sustenta esses quatro blocos reúne mais de oitenta fontes verificáveis, combinando documentação oficial de fornecedor (ANTHROPIC, 2026; MODEL, 2026), catálogos de ataque de organismos de padronização em segurança (OWASP, 2026; CLOUD, 2026), pré-publicações científicas indexadas em arXiv (ARXIV, 2026), relatos de prática de equipes técnicas de mercado (DEPLOYHQ, 2026; SPACELIFT, 2026; TEAMVOY, 2026) e análises técnicas independentes publicadas por profissionais individuais (WILLISON, 2026; KONISHI, 2026).

2.2 Critério de seleção e recuperação das fontes

A seleção das fontes que compõem cada seção deste artigo não foi feita por leitura integral do dossiê-mãe, mas por recuperação indexada: o dossiê foi previamente segmentado em blocos temáticos e indexado por relevância textual (TF-IDF), permitindo consultas pontuais por termos associados a cada um dos quatro pilares do recorte — por exemplo, “settings.json hooks permissions harness configuração”, “servidores MCP schemas tool poisoning blindagem segurança”, “economia de tokens caveman RTK-memory lean-ctx headroom contexto” e “CI/CD deploy agentes portão de aprovação humana pipeline”. Esse procedimento de recuperação seletiva evita carregar o dossiê inteiro em qualquer etapa de síntese e concentra a leitura nos blocos de maior escore de relevância para cada seção IMRaD (Introdução, Metodologia, Resultados e Discussão, Conclusão), preservando a rastreabilidade entre cada afirmação do texto e a fonte original consultada (ANTHROPIC, 2026).

2.3 Procedimento de síntese analítica

A partir dos blocos recuperados, o procedimento de síntese seguiu três etapas. Primeiro, o agrupamento das fontes por camada funcional dentro de cada pilar — por exemplo, dentro do bloco de harness, distinguindo fontes que descrevem o arquivo de configuração propriamente dito (EXPLAINX, 2026) das que descrevem a arquitetura de segurança multicamadas que o envolve (GENERAL, 2026). Segundo, a triangulação entre fontes de natureza distinta — documentação primária de fornecedor, catálogo de ataque de organismo independente e relato de incidente ou prática de mercado — para reduzir o risco de viés de uma única fonte ao descrever um mesmo fenômeno, como ocorre na convergência entre OWASP (2026), Microsoft (2026) e Willison (2026) sobre o mesmo padrão de vulnerabilidade de tool poisoning em MCP. Terceiro, a redação de cada seção com citação autor-data densa (NBR 10520), de modo que toda afirmação analítica remeta a uma fonte identificável do corpus, sem introdução de dado, autor ou ano que não conste do dossiê-mãe já indexado.

2.4 Limites do método

Como recorte derivado, este artigo herda as limitações do dossiê-mãe: não há coleta primária de dados de campo, não há experimento controlado comparando configurações de harness ou arquiteturas de MCP, e a maior parte das fontes é datada de 2026, refletindo a velocidade de publicação de um campo técnico em rápida mudança — o que exige leitura do recorte como fotografia de um estado da arte específico, não como conclusão definitiva. Adicionalmente, a natureza autor-data das citações privilegia a atribuição institucional (por exemplo, ANTHROPIC, 2026; MICROSOFT, 2026) sobre a atribuição individual quando a fonte primária é documentação corporativa sem autoria nominal — limitação inerente ao próprio corpus disponível, não uma escolha editorial deste recorte.

3 RESULTADOS E DISCUSSÃO

3.1 Harness: settings.json, hooks e permissions como camadas de controle

O arquivo `.claude/settings.json` funciona como painel único de decisão do harness: controla qual modelo roda, quais comandos de shell são permitidos, quais servidores MCP se conectam, quais hooks disparam em edições de arquivo e quais variáveis de ambiente são injetadas em cada chamada de shell (EXPLAINX, 2026). Permissões são expressas como arrays `allow`, `deny` e `ask`, com padrões granulares como `Bash(git add:*)`, `WebSearch` ou `SlashCommand(/run-prompt:*)` — a aplicação decide o que é permitido antes de qualquer execução, independentemente do que o modelo tenta (KONISHI, 2026). Hooks são definidos em três níveis de aninhamento: um evento ao qual responder (`PreToolUse`, `Stop`), um grupo de correspondência (*matcher*) que filtra quando o hook dispara e um ou mais manipuladores que

rodam na correspondência, recebendo a entrada via stdin (hooks de comando) ou como corpo de requisição POST (hooks HTTP) (EXPLAINX, 2026).

A camada de hooks resolve um problema que permissions sozinhas não resolvem: confiar em um `deny` de padrão de string exata como antepara final ignora que variações de espaçamento, encadeamento de comandos ou um alias de shell podem produzir um comando funcionalmente idêntico sem bater no padrão declarado — só um hook que inspeciona o comando resolvido no momento da execução corrige essa lacuna de fato (GENERAL, 2026). Por isso, a literatura descreve a segurança do harness como estrutura multicamadas: permissions como aplicação diária, *managed settings* como política corporativa, hooks como aplicação determinística e controles de MCP como governança de ferramentas, na mesma lógica de “acesso mínimo, observação constante, dupla checagem” com que se trataria um agente de IA como um novo funcionário júnior com acesso root (GENERAL, 2026). Guias independentes de arquitetura de harness convergem no mesmo ponto: o harness decide o que é permitido, o modelo decide apenas o que tentar, e a diferença entre as duas coisas é onde vive todo o risco operacional (MINDSTUDIO, 2026; AIMULTIPLE, 2026).

3.2 Tools e servidores MCP: schemas e blindagem contra tool poisoning

Ferramentas no padrão de *tool use* da Claude API expõem um `input_schema` em JSON Schema; quando o modelo decide usar uma ferramenta, retorna um `tool_use` e a aplicação executa a operação, devolvendo um `tool_result` (ANTHROPIC, 2026). Para servidores MCP construídos em FastMCP (Python) ou no MCP SDK (Node/TypeScript), a orientação predominante é equilibrar cobertura abrangente de endpoints de API com ferramentas de fluxo de trabalho especializadas, evitando expor um número excessivo de operações granulares que aumentam a superfície de decisão — e de ataque — do agente (MODEL, 2026). Validação de todas as saídas de chamadas de função antes da execução e *schema validation* para capturar incompatibilidades de tipo são práticas obrigatórias; *rate limiting* previne chamadas de função descontroladas, e operações sensíveis exigem aprovação humana ou regras de validação determinísticas independentes do raciocínio do modelo (APTIBLE, 2026).

O *MCP Tool Poisoning* é o ataque de referência documentado nesse domínio: diferente da injeção de prompt tradicional, a poluição de ferramentas embute instruções diretamente na descrição da ferramenta, injetando-as no contexto do modelo durante a fase de registro do MCP e influenciando a decisão do agente antes de qualquer chamada real (OWASP, 2026). A Microsoft descreve o mesmo padrão como vulnerabilidade estrutural do protocolo, recomendando validação de proveniência de servidores MCP e sandboxing de execução (MICROSOFT, 2026). Willison (2026) argumenta que a combinação, num único agente, de acesso a dados privados, exposição a conteúdo não confiável e capacidade de comunicação externa constitui uma “trifecta letal” que nenhum schema de ferramenta, por si só, neutraliza — a defesa exige também isolamento de rede, permissões mínimas por ferramenta e revisão humana de descrições de MCP antes da instalação, prática recomendada tanto pela Cloud Security Alliance quanto

por relatos independentes de exploração real de chamadas de função em agentes de produção (CLOUD, 2026; SENTRY, 2026).

3.3 Economia severa de tokens como disciplina de contexto

A terceira camada de resultados trata de um problema distinto, mas estruturalmente acoplado às duas primeiras: o custo de operar agentes de codificação em escala é dominado pelo processamento de contexto, não pela geração de texto em si — quanto mais específica a tarefa, mais contexto estranho pode ser cortado, princípio que fundamenta técnicas de compressão como `headroom` (compressão de logs e saídas de comando) e `caveman` (comunicação telegráfica sem perda de precisão técnica) (ANTHROPIC, 2026). O framework acadêmico *SkillReducer* propõe otimizar *skills* de agentes especificamente para eficiência de token, e o paper “The Efficiency Frontier” formaliza um framework unificado de otimização custo-desempenho para gerenciamento de contexto em LLMs — ambos sustentam teoricamente práticas como `rtk-memory`, o registro de padrões e erros para evitar retrabalho de descoberta (ARXIV, 2026).

Sobre busca em código, a literatura observa que, durante a fase de exploração, um agente precisa de geração de hipóteses de cobertura ampla, não de resolução precisa de símbolos: `grep` retorna um cluster de conceitos a partir do qual o modelo infere organização de repositório e convenções de nomenclatura, com custo de token pago a cada resultado despejado na janela de contexto — o princípio que fundamenta `lean-ctx` (`grep` antes de leitura integral) (AGENTA, 2026). O *context engineering* trata a janela de contexto como disciplina central: retrieval ranqueado, filtragem de distintividade semântica e *compaction* de contexto — sumarizar histórico quando a sessão se aproxima do limite da janela, preservando detalhes críticos e descartando saídas redundantes — são técnicas já em uso na própria engenharia de agentes de longa duração (ANTHROPIC, 2026; REDIS, 2026). O paralelo com as camadas anteriores é direto: assim como `permissions` e `hooks` formam anteparas redundantes contra execução indevida, e `schemas` e revisão humana formam anteparas contra `tool poisoning`, as técnicas de economia de tokens formam anteparas contra o colapso econômico de sessões agênticas estendidas — sem elas, o mesmo agente que hoje é auditável e seguro se torna operacionalmente inviável em escala (TOTALUM, 2026).

3.4 Do zero ao deploy: portão de aprovação humana no CI/CD

A quarta camada fecha o ciclo no ponto de maior consequência: o deploy em produção. Práticas de segurança recomendadas incluem credenciais de curta duração e privilégio mínimo para agentes, limite de gasto de tokens, e a manutenção de um portão de aprovação humana entre as mudanças de código do agente e o deploy em produção — o agente abre o *pull request*, o CI valida, um humano aprova o *merge*, e o pipeline de deploy dispara automaticamente; o agente nunca faz deploy direto em produção sem revisão humana (DEPLOYHQ, 2026). Relatos documentados de equipes técnicas — DeployHQ, Spacelift e Teamvoy — descrevem pipelines reais com agentes revisando *pull requests*, reparando testes e disparando remediação

de segurança, sempre com esse portão de aprovação antes de produção (SPACELIFT, 2026; TEAMVOY, 2026).

Os riscos que justificam esse portão não são hipotéticos: alucinação de correções, repetição de ações, comportamento não determinístico e introdução de vulnerabilidades de segurança tornam testes em sandbox, limiares de confiança, *guardrails* operacionais e monitoramento contínuo essenciais, não opcionais (RESEARCHGATE, 2026). O paper “GitInject” documenta ataques reais de injeção de prompt em pipelines de CI/CD alimentados por IA, explorando dados de repositório — títulos de *pull request*, *issues*, comentários — como vetor, o que reforça que o mesmo raciocínio de blindagem contra tool poisoning discutido na Seção 3.2 se aplica, sem exceção, aos metadados que o pipeline de CI/CD injeta no contexto do agente (ARXIV, 2026). Adoção corporativa documentada — Fujitsu automatizando o SDLC completo, e um caso de referência combinando Azure e GitHub Actions — reforça que esse padrão de portão de aprovação humana já é prática de mercado, não recomendação teórica isolada (FUJITSU, 2026; FORRESTER, 2026).

3.5 Síntese integrativa

Lidos em conjunto, os quatro pilares formam uma única pilha de governança: harness (permissions e hooks) decide o que é permitido na sessão; tools e MCP (schemas e blindagem) decidem o que é seguro invocar; economia de tokens decide o que é sustentável manter em contexto; e o portão de CI/CD decide o que é aceitável levar a produção. Nenhuma camada substitui a outra — a ausência de qualquer uma delas reabre exatamente a lacuna que as demais foram desenhadas para fechar, achado que se sustenta de forma consistente em toda a literatura revisada neste recorte (HUMANLAYER, 2026; GENERAL, 2026).

4 CONCLUSÃO

Este recorte investigativo mostrou que a governança de agentes de codificação não se resolve por uma única técnica, mas pela composição de quatro camadas complementares. A primeira é a configuração explícita do harness via `settings.json`, hooks e permissions (EXPLAINX, 2026). A abordagem multicamadas dessa configuração é descrita em detalhe por General (2026). A segunda é a construção de tools e servidores MCP com schemas validados e blindagem ativa contra tool poisoning (OWASP, 2026), problema documentado de forma independente por Microsoft (2026) e também por Willison (2026). A terceira é a disciplina de economia severa de tokens como pré-condição de sustentabilidade operacional em sessões estendidas (ANTHROPIC, 2026), sustentada teoricamente pelo corpus indexado em Arxiv (2026). A quarta é o portão de aprovação humana obrigatório entre a mudança de código proposta pelo agente e o deploy em produção (DEPLOYHQ, 2026), padrão confirmado na prática por Teamvoy (2026). A convergência entre documentação primária de fornecedor, catálogos de ataque de organismos independentes e relatos de prática de mercado sustenta a mesma conclusão a partir de ângulos

distintos: nenhuma dessas camadas, isoladamente, neutraliza os riscos documentados de alucinação, injeção de prompt indireta e explosão de custo de contexto que acompanham a adoção acelerada de IA agêntica em desenvolvimento de software, tendência de mercado quantificada por Futurum (2026) e por Forrester (2026).

A principal implicação prática deste recorte é que equipes que adotam harnesses agênticos como Claude Code devem tratar as quatro camadas como um requisito único de arquitetura, não como itens de configuração independentes a serem endereçados em momentos separados do ciclo de maturidade — a antepara mais fraca determina o nível de exposição real da esteira inteira, independentemente de quão bem configuradas estejam as demais.

REFERÊNCIAS

AGENTA, 2026. *Top techniques to Manage Context Lengths in LLMs*. Disponível em: <https://agenta.ai/blog/top-6-techniques-to-manage-context-length-in-llms>. Acesso em: 02 ago. 2026.

AIMULTIPLE, 2026. *Top Agent Harnesses: Claude Code vs Codex*. Disponível em: <https://aimultiple.com/agent-harness>. Acesso em: 02 ago. 2026.

ANTHROPIC, 2026. *Effective harnesses for long-running agents*. Disponível em: <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>. Acesso em: 02 ago. 2026.

APTIBLE, 2026. *Prompt Injection in MCP: Tool Poisoning and Blast Radius*. Disponível em: <https://www.aptible.com/mcp-security/mcp-prompt-injection>. Acesso em: 02 ago. 2026.

ARXIV, 2026. *GitInject: Real-World Prompt Injection Attacks in AI-Powered CI/CD Pipelines*. Disponível em: <https://arxiv.org/pdf/2606.09935>. Acesso em: 02 ago. 2026.

AUGMENT, 2026. *How to Set Up AI Code Review in Your CI/CD Pipeline*. Augment Code. Disponível em: <https://www.augmentcode.com/guides/ai-code-review-ci-cd-pipeline>. Acesso em: 02 ago. 2026.

CLOUD, 2026. *Agentic MCP Security Best Practices Guide*. Cloud Security Alliance. Disponível em: <https://labs.cloudsecurityalliance.org/agentic/agentic-mcp-security-best-practices-v1/>. Acesso em: 02 ago. 2026.

DEPLOYHQ, 2026. *AI Agents in CI/CD Pipelines: From GitHub Issue to Production Deploy*. Disponível em: <https://www.deployhq.com/blog/ai-agents-cicd-pipelines-github-issue-to-production-deploy>. Acesso em: 02 ago. 2026.

EXPLAINX, 2026. *Claude Code settings.json: Every Option Explained*. Disponível em: <https://www.explainx.ai/blog/claude-code-settings-json-complete-reference-2026>. Acesso em: 02 ago. 2026.

FORRESTER, 2026. *Agentic Software Development Takes The Lead: From Code Assistants To Orchestrated SDLC Agents*. Disponível em: <https://www.forrester.com/blogs/agentic-software-development-takes-the-lead-from-code-assistants-to-orchestrated-sdlc-agents/>. Acesso em: 02 ago. 2026.

FUJITSU, 2026. *Fujitsu automates entire software development lifecycle with new AI-Driven Software Development Platform*. Disponível em: <https://global.fujitsu/en-global/pr/news/2026/02/17-01>. Acesso em: 02 ago. 2026.

FUTURUM, 2026. *AI Reaches 97% of Software Development Organizations*. Futurum Group. Disponível em: <https://futurumgroup.com/press-release/ai-reaches-97-of-software-development-organizations/>. Acesso em: 02 ago. 2026.

GENERAL, 2026. *Anthropic Claude Code Security Best Practices: Permissions, Hooks, MCP, Sandboxing, and CI/CD*. General Analysis. Disponível em: <https://generalanalysis.com/guides/anthropic-claude-code-security-best-practices>. Acesso em: 02 ago. 2026.

HUMANLAYER, 2026. *12-Factor Agents — Principles for Building Reliable LLM Applications*. Disponível em: <https://www.humanlayer.dev/12-factor-agents>. Acesso em: 02 ago. 2026.

KONISHI, Hidekazu, 2026. *Claude Code Features and Settings Reference 2026*. Disponível em: https://hidekazu-konishi.com/entry/claude_code_features_settings_reference_2026.html. Acesso em: 02 ago. 2026.

MICROSOFT, 2026. *Protecting against indirect prompt injection attacks in MCP*. Disponível em: <https://developer.microsoft.com/blog/protecting-against-indirect-injection-attacks-mcp/>. Acesso em: 02 ago. 2026.

MINDSTUDIO, 2026. *What Is an Agent Harness? The Architecture Behind Claude Code, Codex, and Cursor*. Disponível em: <https://www.mindstudio.ai/blog/what-is-agent-harness-architecture-explained>. Acesso em: 02 ago. 2026.

MODEL, 2026. *Specification and documentation for the Model Context Protocol*. Model Context Protocol. Disponível em: <https://github.com/modelcontextprotocol/modelcontextprotocol>. Acesso em: 02 ago. 2026.

OWASP, 2026. *MCP Tool Poisoning*. Disponível em: https://owasp.org/www-community/attacks/MCP_Tool_Poisoning. Acesso em: 02 ago. 2026.

REDIS, 2026. *Context Window Overflow in 2026: Fix LLM Errors Fast*. Disponível em: <https://redis.io/blog/context-window-overflow/>. Acesso em: 02 ago. 2026.

RESEARCHGATE, 2026. *AI-First Software Development Lifecycle: An Agent-Driven Framework for Autonomous Planning, Coding, Testing, and Deployment*. Disponível em: https://www.researchgate.net/publication/403670772_AI-First_Software_Development_Lifecycle_An_Agent-Driven_Framework_for_Autonomous_Planning_Coding_Testing_and_Deployment. Acesso em: 02 ago. 2026.

SENTRY, 2026. *Exploiting Tool and Function Calling in LLM Agents*. Disponível em: <https://blog.sentry.security/exploiting-tool-and-function-calling-in-llm-agents/>. Acesso em: 02 ago. 2026.

SPACELIFT, 2026. *Where Do AI Agents Fit in CI/CD Pipelines?*. Disponível em: <https://spacelift.io/blog/agentic-cicd>. Acesso em: 02 ago. 2026.

TEAMVOY, 2026. *AI Agents in CI/CD Pipelines: A Guide for Tech Leads*. Disponível em: <https://teamvoy.com/blog/building-ai-agents-into-your-ci-cd-pipeline-a-playbook-for-tech-leads/>. Acesso em: 02 ago. 2026.

TOTALUM, 2026. *Claude Code subagents: the 2026 production playbook*. Disponível em: <https://www.totalum.app/blog/claude-code-subagents-totalum>. Acesso em: 02 ago. 2026.

WILLISON, Simon, 2026. *Model Context Protocol has prompt injection security problems*. Disponível em: <https://simonwillison.net/2025/Apr/9/mcp-prompt-injection/>. Acesso em: 02 ago. 2026.